

Apuntes de Clase

Persistencia de datos en Android

De memoria local a backend usando el patrón Repository

Práctica de Clase

Jueves TN. Campus Montserrat

Materia: Desarrollo de Aplicaciones I – Lenguajes Visuales

Resumen

Este apunte desarrolla las principales alternativas de persistencia en Android: estado en memoria, **SharedPreferences**, **DataStore**, archivos de texto, almacenamiento interno y externo, SQLite, Room y backend. Además, organiza esas opciones mediante el patrón *Repository*, con el objetivo de separar la interfaz de usuario de las fuentes concretas de datos.

Índice

1. Introducción: ¿qué significa persistir datos?	4
2. Persistencia en memoria	4
2.1. Usos adecuados	5
2.2. Usos no adecuados	5
3. SharedPreferences	5
3.1. Ventajas	6
3.2. Desventajas	6
4. DataStore como reemplazo moderno de SharedPreferences	7
5. Archivos de texto	8
5.1. Ventajas	8
5.2. Desventajas	9
6. Almacenamiento interno	9
7. Almacenamiento externo	10
7.1. Almacenamiento externo específico de la app	10

7.2. Archivos compartidos con el usuario	11
8. SQLite directo	11
8.1. Ventajas	13
8.2. Desventajas	13
9. SQL Injection en Android	13
10.Room: persistencia local moderna sobre SQLite	14
10.1. Entity	14
10.2. DAO	15
10.3. Database	15
11.Persistencia en backend	16
11.1. Ventajas del backend	17
11.2. Desventajas del backend	17
12.¿Dónde guardar cada tipo de dato?	17
13.Patrón Repository	18
14.DataSource y Repository	19
15.Ejemplo con Repository en memoria	19
16.Repository con SharedPreferences	20
17.Repository con archivo de texto	21
18.Repository con Room	22
19.Repository con backend	24
20.Repository híbrido: Room + Backend	25
21.Repository y Compose	26
22.Seguridad en persistencia	27
23.Comparación general de mecanismos	28

24. Criterios de elección	29
25. Cierre	29

1 Introducción: ¿qué significa persistir datos?

Persistir datos significa guardar información para que no se pierda cuando termina la ejecución de la aplicación, cuando se cierra una pantalla, cuando se rota el dispositivo o incluso cuando el usuario apaga el teléfono.

En una aplicación Android, no todos los datos tienen la misma naturaleza. Algunos datos solo viven mientras la app está abierta. Otros deben guardarse entre sesiones. Otros deben sincronizarse con un servidor. Otros deben quedar disponibles incluso sin conexión.

Por eso, Android ofrece distintas formas de persistencia:

- Datos en memoria.
- Preferencias simples.
- Archivos de texto.
- Almacenamiento interno.
- Almacenamiento externo.
- Base de datos local con SQLite o Room.
- Persistencia remota en backend.

La decisión técnica no debería ser “uso lo que conozco”, sino “uso el mecanismo adecuado para el tipo de dato, volumen, sensibilidad, disponibilidad y vida útil de la información”.

Idea central

La persistencia no es solamente guardar datos. Es decidir **dónde, cómo, por cuánto tiempo, con qué nivel de seguridad y con qué arquitectura** se guarda la información.

2 Persistencia en memoria

La forma más básica de conservar datos durante la ejecución de una app es la memoria. En Jetpack Compose, por ejemplo, se puede usar:

```
var nombre by remember { mutableStateOf("") }
```

Esto conserva el valor mientras el composable permanece en composición. Si cambia el estado, Compose recompone la interfaz.

Ejemplo:

```
@Composable
fun PantallaContador() {
    var contador by remember { mutableStateOf(0) }

    Column {
        Text(text = "Contador: $contador")
    }
}
```

```
        Button(onClick = { contador++ }) {  
            Text("Incrementar")  
        }  
    }  
}
```

Este dato vive en memoria. Si la app se cierra, el dato se pierde. Si la pantalla sale de composición, también puede perderse.

Para conservar datos ante ciertos cambios de configuración, como la rotación de pantalla, se puede usar:

```
var contador by rememberSaveable { mutableStateOf(0) }
```

`rememberSaveable` guarda valores simples usando el mecanismo de estado guardado de Android. Es útil para estados de UI, pero no debe confundirse con persistencia real de negocio.

2.1 Usos adecuados

- Campo de texto temporal.
- Estado de un formulario.
- Pestaña seleccionada.
- Valor momentáneo de una pantalla.

2.2 Usos no adecuados

- Guardar usuarios.
- Guardar facturas.
- Guardar tokens sensibles.
- Guardar historial permanente.
- Guardar datos importantes de negocio.

Pedagógicamente, se puede resumir así:

```
remember           -> memoria de la composición  
rememberSaveable  -> memoria recuperable ante recreación simple  
persistencia real -> archivo, preferencias, base local o backend
```

3 SharedPreferences

`SharedPreferences` es un mecanismo clásico de Android para guardar pares clave-valor simples. Sirve para datos pequeños, de configuración o preferencias del usuario.

Ejemplos típicos:

- Tema oscuro activado.
- Idioma seleccionado.
- Último usuario recordado.
- Flag de primera ejecución.
- Configuración de notificaciones.

Ejemplo con `SharedPreferences`:

```
class PreferenciasUsuario(private val context: Context) {  
  
    private val prefs = context.getSharedPreferences(  
        "preferencias_usuario",  
        Context.MODE_PRIVATE  
    )  
  
    fun guardarNombre(nombre: String) {  
        prefs.edit()  
            .putString("nombre", nombre)  
            .apply()  
    }  
  
    fun obtenerNombre(): String {  
        return prefs.getString("nombre", "") ?: ""  
    }  
}
```

Uso desde una pantalla o `ViewModel`:

```
val preferencias = PreferenciasUsuario(context)  
  
preferencias.guardarNombre("Ana")  
  
val nombre = preferencias.obtenerNombre()
```

3.1 Ventajas

- Simple.
- Rápido de usar.
- Bueno para valores pequeños.
- No requiere diseñar una base de datos.

3.2 Desventajas

- No sirve para datos complejos.

- No maneja bien relaciones.
- No es ideal para listas grandes.
- No es una solución moderna para flujos reactivos.
- No debe usarse como base de datos.

Criterio de ingeniería

Un error común es guardar objetos completos serializados como JSON dentro de **SharedPreferences**. Técnicamente puede hacerse, pero no es recomendable para datos estructurados importantes. Si aparecen listas, relaciones, búsquedas, filtros o actualizaciones parciales, probablemente corresponde usar Room.

4 DataStore como reemplazo moderno de SharedPreferences

Aunque es útil conocer **SharedPreferences**, en aplicaciones modernas conviene presentar **DataStore** como evolución natural para preferencias simples.

DataStore permite guardar datos clave-valor o datos tipados mediante Protocol Buffers. Funciona con coroutines y Flow, por lo que se integra mejor con arquitecturas reactivas y con Compose.

Ejemplo conceptual con Preferences DataStore:

```
val Context.dataStore by preferencesDataStore(name = "settings")

class SettingsRepository(private val context: Context) {

    private val DARK_MODE = booleanPreferencesKey("dark_mode")

    val darkMode: Flow<Boolean> = context.dataStore.data
        .map { preferences ->
            preferences[DARK_MODE] ?: false
        }

    suspend fun setDarkMode(enabled: Boolean) {
        context.dataStore.edit { preferences ->
            preferences[DARK_MODE] = enabled
        }
    }
}
```

Esto permite observar cambios:

```
val darkMode by settingsRepository.darkMode.collectAsState(initial = false)
```

Desde el punto de vista pedagógico, **SharedPreferences** sirve para entender el concepto inicial de clave-valor, pero **DataStore** permite mostrar una solución más alineada con Kotlin, coroutines, Flow y arquitecturas reactivas.

5 Archivos de texto

Otra forma de persistencia es guardar información en archivos. Puede ser texto plano, JSON, CSV, XML, binario o cualquier formato definido por la aplicación.

Un archivo de texto puede ser útil cuando la información tiene forma documental o cuando se quiere exportar/importar datos.

Ejemplos:

- Logs simples.
- Configuración exportable.
- Notas del usuario.
- Datos en CSV.
- Respaldo local.
- Archivo JSON.

Ejemplo de escritura en almacenamiento interno:

```
class ArchivoTextoDataSource(private val context: Context) {  
  
    fun guardarTexto(nombreArchivo: String, contenido: String) {  
        context.openFileOutput(nombreArchivo, Context.MODE_PRIVATE).use { output ->  
            output.write(contenido.toByteArray())  
        }  
    }  
  
    fun leerTexto(nombreArchivo: String): String {  
        return context.openFileInput(nombreArchivo).bufferedReader().use { reader ->  
            reader.readText()  
        }  
    }  
}
```

Uso:

```
val dataSource = ArchivoTextoDataSource(context)  
  
dataSource.guardarTexto(  
    nombreArchivo = "nota.txt",  
    contenido = "Este es un archivo de prueba"  
)  
  
val contenido = dataSource.leerTexto("nota.txt")
```

5.1 Ventajas

- Simple.

- Flexible.
- Bueno para documentos.
- Puede servir para exportar información.

5.2 Desventajas

- No permite consultas eficientes.
- No maneja relaciones.
- No hay integridad referencial.
- No es cómodo para actualizaciones parciales.
- No es ideal para datos complejos.

Un archivo es buena opción cuando se quiere guardar “un documento” o “un bloque de datos”. No es buena opción cuando se quiere manejar entidades, relaciones y búsquedas.

6 Almacenamiento interno

El almacenamiento interno es un espacio privado de la app. En condiciones normales, otras aplicaciones no pueden acceder a esos archivos. Además, cuando la app se desinstala, esos datos se eliminan.

Ejemplo:

```
fun guardarEnInterno(context: Context, nombreArchivo: String, contenido: String) {
    context.openFileOutput(nombreArchivo, Context.MODE_PRIVATE).use { output ->
        output.write(contenido.toByteArray())
    }
}
```

Lectura:

```
fun leerDesdeInterno(context: Context, nombreArchivo: String): String {
    return context.openFileInput(nombreArchivo).bufferedReader().use {
        it.readText()
    }
}
```

También se puede acceder al directorio interno de archivos:

```
val archivo = File(context.filesDir, "datos.txt")
archivo.writeText("Contenido guardado")
```

Lectura:

```
val archivo = File(context.filesDir, "datos.txt")
val contenido = archivo.readText()
```

Casos adecuados:

- Datos propios de la app.
- Archivos que no deberían ser visibles al usuario.
- Caché controlada.
- Información auxiliar.
- Archivos de configuración internos.
- Datos no compartibles.

Advertencia de seguridad

“Interno” no significa “criptográficamente protegido”. En dispositivos comprometidos, con root, backups inseguros o mala configuración, puede haber exposición. Si el dato es sensible, hay que considerar cifrado, Android Keystore y políticas de backup.

7 Almacenamiento externo

El almacenamiento externo puede ser *app-specific* o compartido. Esto suele confundir, porque “externo” no siempre significa “tarjeta SD removable”. En Android moderno, *external storage* puede ser una zona de almacenamiento accesible bajo ciertas reglas del sistema.

7.1 Almacenamiento externo específico de la app

Se obtiene con:

```
context.getExternalFilesDir(null)
```

Ejemplo:

```
fun guardarEnExternoApp(context: Context, nombreArchivo: String, contenido: String)
↳ {
    val dir = context.getExternalFilesDir(null)
    val archivo = File(dir, nombreArchivo)
    archivo.writeText(contenido)
}
```

Lectura:

```
fun leerDesdeExternoApp(context: Context, nombreArchivo: String): String {  
    val dir = context.getExternalFilesDir(null)  
    val archivo = File(dir, nombreArchivo)  
    return archivo.readText()  
}
```

Este tipo de almacenamiento es útil para archivos propios de la app pero que pueden ocupar más espacio o no requieren la misma privacidad que el almacenamiento interno.

Casos posibles:

- Imágenes generadas por la app.
- Archivos temporales grandes.
- Exportaciones.
- Documentos que el usuario podría querer recuperar.

7.2 Archivos compartidos con el usuario

Cuando el archivo debe ser visible para el usuario o compartible con otras apps, se deberían usar APIs específicas:

- **MediaStore**, para imágenes, audio y video.
- **Storage Access Framework**, para documentos seleccionados por el usuario.

No conviene escribir arbitrariamente en cualquier ruta del sistema. Android moderno restringe mucho más el acceso al almacenamiento compartido por razones de seguridad y privacidad.

8 SQLite directo

SQLite es una base de datos relacional embebida. Android la soporta desde hace muchos años. Permite crear tablas, insertar registros, consultar con SQL y manejar datos estructurados.

Ejemplo conceptual:

```
CREATE TABLE usuarios (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nombre TEXT NOT NULL,  
    email TEXT NOT NULL  
);
```

En Android se puede trabajar con SQLite usando clases como **SQLiteOpenHelper**, pero hoy no suele ser la opción recomendada para aplicaciones modernas si se puede usar Room.

Ejemplo simplificado:

```
class MiDbHelper(context: Context) :  
    SQLiteOpenHelper(context, "app.db", null, 1) {  
  
    override fun onCreate(db: SQLiteDatabase) {  
        db.execSQL(  
            """  
            CREATE TABLE usuarios (  
                id INTEGER PRIMARY KEY AUTOINCREMENT,  
                nombre TEXT NOT NULL,  
                email TEXT NOT NULL  
            )  
            """  
        )  
    }  
  
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {  
        db.execSQL("DROP TABLE IF EXISTS usuarios")  
        onCreate(db)  
    }  
}
```

Insertar:

```
fun insertarUsuario(db: SQLiteDatabase, nombre: String, email: String) {  
    val valores = ContentValues().apply {  
        put("nombre", nombre)  
        put("email", email)  
    }  
  
    db.insert("usuarios", null, valores)  
}
```

Consultar:

```
fun obtenerUsuarios(db: SQLiteDatabase): List<String> {  
    val usuarios = mutableListOf<String>()  
  
    val cursor = db.query(  
        "usuarios",  
        arrayOf("nombre", "email"),  
        null,  
        null,  
        null,  
        null,  
        null  
    )  
  
    cursor.use {  
        while (it.moveToNext()) {  
            val nombre = it.getString(0)  
            usuarios.add(nombre)  
        }  
    }  
}
```

```
        val email = it.getString(1)
        usuarios.add("$nombre - $email")
    }
}

return usuarios
}
```

8.1 Ventajas

- Potente.
- Relacional.
- Permite SQL real.
- No depende de servidor.

8.2 Desventajas

- Más verboso.
- Más propenso a errores.
- Hay que manejar migraciones manualmente.
- No tiene verificaciones fuertes en tiempo de compilación.
- Puede generar mucho código repetitivo.

9 SQL Injection en Android

Cuando se habla de SQLite, hay que hablar de SQL Injection. No porque sea exclusivo de Android, sino porque cualquier uso incorrecto de SQL puede abrir vulnerabilidades.

Un ejemplo peligroso sería:

```
val query = "SELECT * FROM usuarios WHERE email = '$email'"
db.rawQuery(query, null)
```

Si `email` viene de entrada del usuario, podría contener caracteres que alteren la consulta.

Ejemplo malicioso:

```
' OR '1'='1
```

La forma correcta es usar parámetros:

```
val cursor = db.rawQuery(  
    "SELECT * FROM usuarios WHERE email = ?",  
    arrayOf(email)  
)
```

O, mejor aún, usar Room con consultas parametrizadas:

```
@Query("SELECT * FROM usuarios WHERE email = :email")  
suspend fun buscarPorEmail(email: String): UsuarioEntity?
```

Regla pedagógica:

- Nunca concatenar entrada del usuario dentro de una consulta SQL.
- Usar parámetros.
- Validar entradas.
- No confiar en datos provenientes de UI, intents, archivos o backend.
- Usar abstracciones seguras cuando sea posible.

Desde una mirada de seguridad aplicada, SQL Injection no desaparece porque la base sea local. Si el dato de entrada puede ser manipulado, la consulta puede ser manipulada. En apps móviles, además, hay que considerar intents, deep links, archivos importados, QR, NFC, respuestas de backend y cualquier canal de entrada.

10 Room: persistencia local moderna sobre SQLite

Room es la opción recomendada de Jetpack para persistencia local estructurada. Room no reemplaza conceptualmente a SQLite: se construye sobre SQLite y ofrece una capa de abstracción más segura y cómoda.

Room trabaja principalmente con tres elementos:

- Entity.
- DAO.
- Database.

10.1 Entity

Una Entity representa una tabla.

```
@Entity(tableName = "usuarios")  
data class UsuarioEntity(  
    @PrimaryKey(autoGenerate = true)
```

```
    val id: Int = 0,  
    val nombre: String,  
    val email: String  
)
```

10.2 DAO

Un DAO define las operaciones sobre la base.

```
@Dao  
interface UsuarioDao {  
  
    @Insert  
    suspend fun insertar(usuario: UsuarioEntity)  
  
    @Query("SELECT * FROM usuarios")  
    fun obtenerTodos(): Flow<List<UsuarioEntity>>  
  
    @Query("SELECT * FROM usuarios WHERE id = :id")  
    suspend fun obtenerPorId(id: Int): UsuarioEntity?  
  
    @Delete  
    suspend fun eliminar(usuario: UsuarioEntity)  
}
```

10.3 Database

La clase RoomDatabase representa la base.

```
@Database(  
    entities = [UsuarioEntity::class],  
    version = 1  
)  
abstract class AppDatabase : RoomDatabase() {  
    abstract fun usuarioDao(): UsuarioDao  
}
```

Creación de instancia:

```
val db = Room.databaseBuilder(  
    context,  
    AppDatabase::class.java,  
    "app_database"  
)build()
```

En una app real, esta instancia no debería crearse en cualquier lugar. Normalmente se centraliza con inyección de dependencias o un contenedor de dependencias.

Ventajas de Room:

- Menos código repetitivo.
- Mejor integración con Kotlin.
- Soporte para coroutines y Flow.
- Validación de consultas en tiempo de compilación.
- Manejo más claro de entidades y DAOs.
- Mejor separación de responsabilidades.

11 Persistencia en backend

La persistencia local no siempre alcanza. Muchas aplicaciones necesitan guardar datos en un servidor.

Ejemplos:

- Usuarios registrados.
- Pedidos.
- Pagos.
- Mensajes.
- Datos compartidos entre dispositivos.
- Sincronización entre web y móvil.
- Auditoría.
- Respaldo centralizado.
- Procesos colaborativos.

Una app Android puede comunicarse con un backend usando HTTP, generalmente mediante REST o GraphQL. En Kotlin/Android es común usar Retrofit, Ktor Client o librerías similares.

Ejemplo conceptual con Retrofit:

```
interface UsuarioApi {  
  
    @GET("usuarios")  
    suspend fun obtenerUsuarios(): List<UsuarioDto>  
  
    @POST("usuarios")  
    suspend fun crearUsuario(  
        @Body usuario: UsuarioDto  
    ): UsuarioDto  
}
```

DTO:


```
data class UsuarioDto(  
    val id: Int,  
    val nombre: String,  
    val email: String  
)
```

11.1 Ventajas del backend

- Persistencia centralizada.
- Datos compartidos entre dispositivos.
- Control del negocio desde servidor.
- Mayor capacidad de auditoría.
- Integración con otros sistemas.
- Sincronización multiusuario.

11.2 Desventajas del backend

- Requiere conectividad.
- Puede fallar por red.
- Mayor latencia.
- Hay que manejar autenticación.
- Hay que proteger la comunicación.
- Hay que diseñar bien errores, reintentos y estados offline.

Desde seguridad, la persistencia en backend introduce nuevas responsabilidades: TLS, autenticación, autorización, validación server-side, protección de tokens, manejo de sesiones, control de errores, logs, rate limiting y diseño seguro de APIs.

12 ¿Dónde guardar cada tipo de dato?

Una forma didáctica de elegir mecanismo es esta:

Tipo de dato	Opción razonable
Estado temporal de pantalla	<code>remember</code> , <code>rememberSaveable</code>
Preferencias simples	<code>DataStore</code> o <code>SharedPreferences</code>
Configuración chica clave-valor	<code>DataStore</code>
Texto o documento simple	Archivo interno
Archivo propio de la app	Almacenamiento interno
Archivo grande propio de la app	External app-specific storage
Imagen/video compartible	<code>MediaStore</code>
Documento elegido por el usuario	Storage Access Framework
Datos estructurados locales	<code>Room</code>

Datos relacionales complejos	Room / SQLite
Datos compartidos entre dispositivos	Backend
Datos que deben sobrevivir reinstalación	Backend
Caché offline	Room + backend
Datos sensibles	Evaluar cifrado, Keystore, backend y threat model

La pregunta correcta no es “¿cuál es mejor?”, sino “¿mejor para qué?”.

13 Patrón Repository

El patrón Repository busca aislar al resto de la aplicación del origen real de los datos.

La UI no debería saber si los datos vienen de:

- Memoria.
- `SharedPreferences`.
- `DataStore`.
- Archivo.
- Room.
- SQLite.
- API remota.
- Caché.
- Backend.

Conceptualmente:

```
UI / Compose
|
v
ViewModel
|
v
Repository
|
v
DataSource local / remoto / archivo / preferencias / base de datos
```

La UI no debería llamar directamente a Room, Retrofit o `SharedPreferences`.

Mal enfoque:

```
Composable -> Room
Composable -> Retrofit
Composable -> SharedPreferences
```

Mejor enfoque:

```
Composable -> ViewModel -> Repository -> DataSource
```

Regla de arquitectura

La UI no debería saber dónde se guardan los datos. La UI debería pedir datos y enviar eventos. El Repository debería resolver de dónde vienen, dónde se guardan y cómo se sincronizan.

14 DataSource y Repository

Conviene distinguir dos responsabilidades.

- El **DataSource** sabe hablar con una fuente concreta.
- El **Repository** decide cómo usar una o varias fuentes de datos.

Ejemplo:

```
UsuarioLocalDataSource -> Room  
UsuarioRemoteDataSource -> Backend  
UsuarioRepository      -> coordina local + remoto
```

Esto permite cambiar implementaciones sin romper toda la app. Por ejemplo, hoy el usuario se guarda en memoria. Mañana en Room. Pasado mañana se sincroniza con backend. Si la UI depende de un repositorio, esos cambios quedan contenidos.

15 Ejemplo con Repository en memoria

Primero, una versión simple en memoria.

Modelo de dominio:

```
data class Usuario(  
    val id: Int,  
    val nombre: String,  
    val email: String  
)
```

Contrato del repositorio:

```
interface UsuarioRepository {  
    fun obtenerUsuarios(): List<Usuario>  
    fun agregarUsuario(usuario: Usuario)  
}
```

Implementación en memoria:

```
class UsuarioMemoryRepository : UsuarioRepository {  
    private val usuarios = mutableListOf<Usuario>()  
  
    override fun obtenerUsuarios(): List<Usuario> {  
        return usuarios  
    }  
  
    override fun agregarUsuario(usuario: Usuario) {  
        usuarios.add(usuario)  
    }  
}
```

Esto no persiste cuando la app se cierra, pero sirve para enseñar la abstracción. La UI no sabe si los datos están en memoria, Room o backend. Solo conoce el contrato.

16 Repository con SharedPreferences

Supongamos que queremos guardar el nombre de usuario.

Contrato:

```
interface PerfilRepository {  
    fun guardarNombre(nombre: String)  
    fun obtenerNombre(): String  
}
```

Implementación:

```
class PerfilSharedPreferencesRepository(  
    context: Context  
) : PerfilRepository {  
  
    private val prefs = context.getSharedPreferences(  
        "perfil",  
        Context.MODE_PRIVATE  
    )  
  
    override fun guardarNombre(nombre: String) {  
        prefs.edit()  
            .putString("nombre", nombre)  
            .apply()  
    }  
  
    override fun obtenerNombre(): String {  
        return prefs.getString("nombre", "") ?: ""  
    }  
}
```

```
}  
}
```

Uso conceptual desde ViewModel:

```
class PerfilViewModel(  
    private val repository: PerfilRepository  
) : ViewModel() {  
  
    var nombre by mutableStateOf(repository.obtenerNombre())  
    private set  
  
    fun actualizarNombre(nuevoNombre: String) {  
        nombre = nuevoNombre  
        repository.guardarNombre(nuevoNombre)  
    }  
}
```

La pantalla Compose solo habla con el ViewModel:

```
@Composable  
fun PantallaPerfil(viewModel: PerfilViewModel) {  
    val nombre = viewModel.nombre  
  
    TextField(  
        value = nombre,  
        onChange = { viewModel.actualizarNombre(it) },  
        label = { Text("Nombre") }  
    )  
}
```

17 Repository con archivo de texto

Contrato:

```
interface NotaRepository {  
    fun guardarNota(texto: String)  
    fun leerNota(): String  
}
```

Implementación:

```
class NotaArchivoRepository(  
    private val context: Context
```

```
) : NotaRepository {  
  
    private val nombreArchivo = "nota.txt"  
  
    override fun guardarNota(texto: String) {  
        context.openFileOutput(nombreArchivo, Context.MODE_PRIVATE).use { output ->  
            output.write(texto.toByteArray())  
        }  
    }  
  
    override fun leerNota(): String {  
        return try {  
            context.openFileInput(nombreArchivo).bufferedReader().use {  
                it.readText()  
            }  
        } catch (e: FileNotFoundException) {  
            ""  
        }  
    }  
}
```

Este repositorio oculta que la nota se guarda en un archivo. La UI no necesita conocer `openFileOutput`, `File`, rutas ni excepciones.

18 Repository con Room

Modelo de dominio:

```
data class Usuario(  
    val id: Int,  
    val nombre: String,  
    val email: String  
)
```

Entity de Room:

```
@Entity(tableName = "usuarios")  
data class UsuarioEntity(  
    @PrimaryKey(autoGenerate = true)  
    val id: Int = 0,  
    val nombre: String,  
    val email: String  
)
```

Mapper:

```
fun UsuarioEntity.toDomain(): Usuario {
```

```
        return Usuario(  
            id = id,  
            nombre = nombre,  
            email = email  
        )  
    }  
  
    fun Usuario.toEntity(): UsuarioEntity {  
        return UsuarioEntity(  
            id = id,  
            nombre = nombre,  
            email = email  
        )  
    }  
}
```

DAO:

```
@Dao  
interface UsuarioDao {  
  
    @Query("SELECT * FROM usuarios")  
    fun observarUsuarios(): Flow<List<UsuarioEntity>>  
  
    @Insert  
    suspend fun insertar(usuario: UsuarioEntity)  
  
    @Delete  
    suspend fun eliminar(usuario: UsuarioEntity)  
}
```

Repository:

```
interface UsuarioRepository {  
    fun observarUsuarios(): Flow<List<Usuario>>  
    suspend fun agregarUsuario(usuario: Usuario)  
}
```

Implementación:

```
class UsuarioRoomRepository(  
    private val usuarioDao: UsuarioDao  
) : UsuarioRepository {  
  
    override fun observarUsuarios(): Flow<List<Usuario>> {  
        return usuarioDao.observeUsuarios()  
            .map { lista ->  
                lista.map { it.toDomain() }  
            }  
    }  
}
```

```
override suspend fun agregarUsuario(usuario: Usuario) {  
    usuarioDao.insertar(usuario.toEntity())  
}  
}
```

Acá aparece una buena práctica importante: no exponer directamente **UsuarioEntity** fuera de la capa de datos. **UsuarioEntity** pertenece a Room; **Usuario** pertenece al dominio de la app. Esto evita que la base de datos contamine toda la aplicación.

19 Repository con backend

DTO remoto:

```
data class UsuarioDto(  
    val id: Int,  
    val nombre: String,  
    val email: String  
)
```

Mapper:

```
fun UsuarioDto.toDomain(): Usuario {  
    return Usuario(  
        id = id,  
        nombre = nombre,  
        email = email  
    )  
}  
  
fun Usuario.toDto(): UsuarioDto {  
    return UsuarioDto(  
        id = id,  
        nombre = nombre,  
        email = email  
    )  
}
```

API:

```
interface UsuarioApi {  
  
    @GET("usuarios")  
    suspend fun obtenerUsuarios(): List<UsuarioDto>  
  
    @POST("usuarios")  
    suspend fun crearUsuario(  

```



```
@Body usuario: UsuarioDto  
) : UsuarioDto  
}
```

Repository remoto:

```
class UsuarioRemoteRepository(  
    private val api: UsuarioApi  
) : UsuarioRepository {  
  
    override fun observarUsuarios(): Flow<List<Usuario>> = flow {  
        val usuarios = api.obtenerUsuarios()  
        .map { it.toDomain() }  
  
        emit(usuarios)  
    }  
  
    override suspend fun agregarUsuario(usuario: Usuario) {  
        api.crearUsuario(usuario.toDto())  
    }  
}
```

Este repositorio obtiene datos desde backend. Pero para la UI el contrato sigue siendo el mismo.

20 Repository híbrido: Room + Backend

Una app real muchas veces usa persistencia local y remota al mismo tiempo.

Ejemplo típico:

1. La app consulta el backend.
2. Guarda los datos en Room.
3. La UI observa Room.
4. Si no hay internet, la UI sigue mostrando los últimos datos locales.
5. Cuando vuelve la red, se sincroniza.

Arquitectura:

```
Compose UI  
|  
v  
ViewModel  
|  
v  
UsuarioRepository  
|  
v
```

Room + API

Ejemplo simplificado:

```
class UsuarioOfflineFirstRepository(
    private val usuarioDao: UsuarioDao,
    private val api: UsuarioApi
) : UsuarioRepository {

    override fun observarUsuarios(): Flow<List<Usuario>> {
        return usuarioDao.observarUsuarios()
            .map { lista ->
                lista.map { it.toDomain() }
            }
    }

    suspend fun sincronizar() {
        val usuariosRemotos = api.obtenerUsuarios()

        usuariosRemotos.forEach { dto ->
            usuarioDao.insertar(dto.toDomain().toEntity())
        }
    }

    override suspend fun agregarUsuario(usuario: Usuario) {
        val creado = api.crearUsuario(usuario.toDto())
        usuarioDao.insertar(creado.toDomain().toEntity())
    }
}
```

Este enfoque suele llamarse *offline-first*: la app prioriza una experiencia funcional incluso sin conexión, usando una base local como fuente observable para la UI.

21 Repository y Compose

En Compose, lo ideal es que la pantalla no acceda directamente al repositorio. Normalmente se usa un ViewModel.

Ejemplo:

```
class UsuarioViewModel(
    private val repository: UsuarioRepository
) : ViewModel() {

    val usuarios: StateFlow<List<Usuario>> =
        repository.observarUsuarios()
            .stateIn(
                scope = viewModelScope,
                started = SharingStarted.WhileSubscribed(5000),
```

```
        initialValue = emptyList()
    )

    fun agregarUsuario(nombre: String, email: String) {
        viewModelScope.launch {
            repository.agregarUsuario(
                Usuario(
                    id = 0,
                    nombre = nombre,
                    email = email
                )
            )
        }
    }
}
```

Composable:

```
@Composable
fun PantallaUsuarios(viewModel: UsuarioViewModel) {
    val usuarios by viewModel.usuarios.collectAsState()

    Column {
        usuarios.forEach { usuario ->
            Text(text = "${usuario.nombre} - ${usuario.email}")
        }

        Button(
            onClick = {
                viewModel.agregarUsuario(
                    nombre = "Ana",
                    email = "ana@example.com"
                )
            }
        ) {
            Text("Agregar")
        }
    }
}
```

La pantalla no sabe si los usuarios vienen de memoria, Room, archivo o backend. Eso es exactamente lo que se busca con Repository.

22 Seguridad en persistencia

En Android, persistir datos no es solamente una cuestión funcional. También es una cuestión de seguridad.

Preguntas importantes:

- ¿El dato es sensible?

- ¿Debe cifrarse?
- ¿Puede quedar en backups?
- ¿Puede exponerse por logs?
- ¿Se guarda en almacenamiento externo?
- ¿Se transmite a backend?
- ¿El backend valida autorización?
- ¿El usuario puede exportarlo?
- ¿Otra app podría leerlo?
- ¿Se borra al cerrar sesión?

Ejemplos:

- No guardar contraseñas en texto claro.
- No guardar tokens sensibles en archivos externos.
- No guardar PAN, CVV o datos regulados sin un análisis normativo serio.
- No confiar en que “está en el teléfono” equivale a “está seguro”.
- No concatenar SQL.
- No guardar información sensible en logs.
- No usar almacenamiento externo para secretos.

Si el dato es sensible, hay que evaluar Android Keystore, cifrado local, expiración de sesión, invalidación de tokens, protección del backend, controles de acceso y threat modeling.

Desde una mirada de ingeniería, persistencia y seguridad no son temas separados. La forma en que se guarda un dato define parte de la superficie de ataque de la aplicación.

23 Comparación general de mecanismos

Mecanismo	Persistente	Estructurado	Compartible	Uso principal
<code>remember</code>	No	No	No	Estado temporal de UI
<code>rememberSaveable</code>	Parcial	No	No	Estado recuperable de pantalla
<code>SharedPreferences</code>	Sí	Poco	No	Preferencias simples
<code>DataStore</code>	Sí	Medio	No	Preferencias modernas
Archivo interno	Sí	Depende	No	Documentos propios
Archivo externo app-specific	Sí	Depende	Parcial	Archivos propios grandes
<code>MediaStore</code>	Sí	No	Sí	Imágenes, audio, video
SAF	Sí	No	Sí	Documentos elegidos por usuario

SQLite directo	Sí	Sí	No	Base relacional local
Room	Sí	Sí	No	Persistencia local moderna
Backend	Sí	Sí	Sí	Datos remotos y sincronizados

24 Criterios de elección

Para elegir una estrategia de persistencia, conviene preguntar:

- **¿Cuánto dato tengo?** Si son pocos valores simples, puede alcanzar con `DataStore` o `SharedPreferences`. Si son entidades con relaciones, corresponde `Room`. Si son documentos, corresponde archivo. Si deben sincronizarse, corresponde backend.
- **¿El dato debe sobrevivir al cierre de la app?** Si no, memoria. Si sí, persistencia local o remota.
- **¿Debe estar disponible offline?** Si sí, `Room` o archivos locales.
- **¿Debe compartirse entre dispositivos?** Si sí, backend.
- **¿Debe ser visible para el usuario?** Si sí, almacenamiento compartido, `MediaStore` o `SAF`.
- **¿Es sensible?** Si sí, revisar cifrado, `Keystore`, permisos, backups y backend.
- **¿Necesito consultar, filtrar u ordenar?** Si sí, `Room` o `SQLite`.
- **¿Necesito sincronización?** Si sí, `Repository` híbrido local-remoto.

25 Cierre

Android ofrece muchas opciones de persistencia porque no todos los datos tienen las mismas necesidades. Para estado temporal de pantalla, alcanza con memoria y estado `Compose`. Para preferencias simples, puede usarse `SharedPreferences`, aunque en aplicaciones modernas conviene `DataStore`. Para documentos o texto plano, se pueden usar archivos internos o externos según si el dato debe ser privado o compartible. Para datos estructurados, relaciones y consultas, `Room` es la opción local más recomendable. Para datos compartidos entre dispositivos, sincronización o lógica centralizada, se necesita backend.

El patrón `Repository` permite ocultar el origen de los datos y ofrecer una interfaz limpia al resto de la aplicación. Gracias a este patrón, la UI no depende de si los datos vienen de memoria, preferencias, archivos, `Room` o backend.

Síntesis final

La UI no debería saber dónde se guardan los datos. La UI debería pedir datos y enviar eventos. El `Repository` debería resolver de dónde vienen, dónde se guardan y cómo se sincronizan.